I used the gemini and claude websites. The first prompt i gave was the rubric, the assignment instructions, and the other students assignment and with the message: “give me the code i need”

Gemini raw code:

```
/*
Program: EECS 348 Assignment 5 - Recursive Sudoku Solver (Modified)
Description: Reads five Sudoku puzzles, writes every valid solution and solution count
             to solution1.txt through solution5.txt using recursive backtracking.
Inputs: puzzle1.txt, puzzle2.txt, puzzle3.txt, puzzle4.txt, puzzle5.txt
Outputs: solution1.txt, solution2.txt, solution3.txt, solution4.txt, solution5.txt
Author: Peter Lauman
Revised by: [Your Full Name]
Created: October 15, 2026
Revision Date: November 2, 2026
Collaborators / sources: OpenAI ChatGPT, EECS 348 Assignment 4 base code
*/

#include <array>      // Fixed-size containers hold the nine-by-nine grid.
#include <fstream>    // Input streams read each puzzle file and output streams write solutions.
#include <iostream>   // Standard output displays messages.
#include <string>     // File names and error messages are strings.
#include <vector>     // Candidate lists store valid numbers for a cell.

class SudokuSolver {
private:
    std::array<std::array<int, 9>, 9> grid{}; // Zero represents an empty square.
    unsigned long long solutions = 0;         // Track how many completed grids are found.

    // Check whether a number violates any Sudoku constraint at a given square.
    bool allowed(int row, int col, int value) const {
        for (int i = 0; i < 9; ++i) { // Scan the corresponding row and column.
            if (grid[row][i] == value || grid[i][col] == value) return false;
        }
        int firstRow = (row / 3) * 3; // Find the top row of the containing block.
        int firstCol = (col / 3) * 3; // Find the left column of the containing block.
        for (int r = firstRow; r < firstRow + 3; ++r) { // Visit each block row.
            for (int c = firstCol; c < firstCol + 3; ++c) { // Visit each block column.
                if (grid[r][c] == value) return false; // Reject repeated digits.
            }
        }
        return true; // No conflict was found.
    }
```

```
// Use the empty square with the fewest legal candidates to reduce branching.
bool chooseCell(int& bestRow, int& bestCol, std::vector<int>& bestValues) const {
    bestRow = -1; // Negative row means the board has no empty cells.
    bestCol = -1; // Negative column also indicates no choice yet.
    bestValues.clear(); // Remove candidates from the previous recursion level.
    for (int r = 0; r < 9; ++r) { // Inspect all rows.
        for (int c = 0; c < 9; ++c) { // Inspect all columns.
            if (grid[r][c] != 0) continue; // Ignore already filled cells.
            std::vector<int> candidates; // Store legal values for this square.
            for (int v = 1; v <= 9; ++v) { // Try digits one through nine.
                if (allowed(r, c, v)) candidates.push_back(v); // Keep legal digits.
            }
            if (candidates.empty()) { // An empty square has no valid digit.
                bestRow = r; // Identify the contradictory square.
                bestCol = c; // Identify its column.
                bestValues.clear(); // Signal that this branch cannot work.
                return true; // A blank cell exists, but this branch is impossible.
            }
            if (bestRow == -1 || candidates.size() < bestValues.size()) { // Prefer fewer options.
                bestRow = r; // Save the strongest next square.
                bestCol = c; // Save its column.
                bestValues = candidates; // Save its available digits.
            }
        }
    }
    return bestRow != -1; // False means the entire board is filled.
}

// Recursively explore every valid completion of the current grid, writing to output stream.
void backtrack(std::ofstream& outFile) {
    int row, col; // Coordinates of the next empty square.
    std::vector<int> values; // Valid candidates for that square.
    if (!chooseCell(row, col, values)) { // No empty squares remain.
        ++solutions; // Count the completed Sudoku board.
        outFile << "Solution " << solutions << ":\n"; // Label each solution in file.
        print(outFile); // Print the completed board to file.
        return; // Continue searching after returning to the caller.
    }
    for (int value : values) { // Explore each candidate in order.
        grid[row][col] = value; // Tentatively fill the chosen square.
        backtrack(outFile); // Recursively solve the smaller puzzle.
        grid[row][col] = 0; // Undo the guess so other possibilities can be tried.
    }
}
```

```
public:
    // Read exactly 81 tokens, allowing underscores for blank squares.
    bool load(const std::string& filename) {
        std::ifstream input(filename); // Open the named puzzle.
        if (!input) return false; // Report missing or unreadable files.
        grid = {}; // Reset the grid before reading another puzzle.
        solutions = 0; // Reset the solution counter.
        std::string token; // Hold each number or underscore.
        for (int r = 0; r < 9; ++r) { // Read nine rows.
            for (int c = 0; c < 9; ++c) { // Read nine cells per row.
                if (!(input >> token)) return false; // Reject short or malformed files.
                if (token == "_") grid[r][c] = 0; // Convert a blank to zero.
                else if (token.size() == 1 && token[0] >= '1' && token[0] <= '9')
                    grid[r][c] = token[0] - '0'; // Convert a digit to an integer.
                else return false; // Reject unexpected input.
            }
        }
        if (input >> token) return false; // Reject files containing extra cells.
        for (int r = 0; r < 9; ++r) { // Check all given values.
            for (int c = 0; c < 9; ++c) { // Check every square in each row.
                if (grid[r][c] == 0) continue; // Blank squares cannot conflict.
                int value = grid[r][c]; // Temporarily remove a given digit.
                grid[r][c] = 0; // Avoid comparing the given with itself.
                bool valid = allowed(r, c, value); // Test row, column, and block.
                grid[r][c] = value; // Restore the original given digit.
                if (!valid) return false; // Reject conflicting clues.
            }
        }
        return true; // The puzzle was read and validated.
    }

    // Print the grid to a provided output stream in the input format.
    void print(std::ostream& out) const {
        for (const auto& row : grid) { // Iterate through the nine rows.
            for (int c = 0; c < 9; ++c) { // Print all nine columns.
                if (c) out << ' '; // Separate adjacent cells with spaces.
                if (row[c] == 0) out << '_'; // Show blanks as underscores.
                else out << row[c]; // Show filled squares as digits.
            }
            out << '\n'; // End the current row.
        }
        out << '\n'; // Separate grids visually.
    }
```

```
    // Solve the puzzle and write results to the specified output file.
    void solve(const std::string& outFilename) {
        std::ofstream outFile(outFilename);
        if (!outFile) {
            std::cerr << "Error opening output file: " << outFilename << "\n";
            return;
        }

        backtrack(outFile); // Begin recursive depth-first search writing to file.

        if (solutions == 0) {
            outFile << "No solution found.\n"; // Handle failure as required.
        } else {
            outFile << "Solutions found: " << solutions << "\n"; // Report count as required.
        }
        outFile.close();
    }
};

// Run the object-oriented solver on all five assigned puzzle files and generate solutionN.txt files.
int main() {
    for (int number = 1; number <= 5; ++number) { // Process every supplied puzzle.
        std::string filename = "puzzle" + std::to_string(number) + ".txt"; // Build input file name.
        std::string outFilename = "solution" + std::to_string(number) + ".txt"; // Build output file
name.

        SudokuSolver sudoku; // Create a fresh object for this puzzle.
        std::cout << "Processing " << filename << " -> writing to " << outFilename << "...\n";

        if (!sudoku.load(filename)) { // Read and validate the puzzle.
            std::cout << "Unable to read puzzle or invalid starting grid for " << filename << ".\n\n";

            // Write "No solution found." to output file if file can't even be loaded properly
            std::ofstream outFile(outFilename);
            if (outFile) {
                outFile << "No solution found.\n";
            }
            continue; // Move on to the next puzzle file.
        }

        sudoku.solve(outFilename); // Solve and output directly to solutionN.txt
    }
    std::cout << "All puzzles processed successfully.\n";
```

```
    return 0; // Finish successfully.
}
```

Claude raw code:

```
/*
Program: EECS 348 Assignment 5 - Recursive Sudoku Solver (writes solutions to files)
Description: Reads five Sudoku puzzles and uses recursive backtracking to find every valid
solution.
             Solutions are printed to the console and written to solution1.txt - solution5.txt.
Inputs: puzzle1.txt, puzzle2.txt, puzzle3.txt, puzzle4.txt, puzzle5.txt
Outputs: Console output plus solutionN.txt for each puzzle N. Each file holds either the single
line
         "No solution found." or all solutions (same format as the puzzle files) followed by
         "Solutions found: n".
Original author: Peter Lauman (Assignment 4)
Original creation date: October 15, 2026
Revised by: Nathaniel Lanter    Revision date: November 2, 2026   <-- set to the real date you
edited
Collaborators / sources: Peter Lauman (original Assignment 4 code); OpenAI ChatGPT (used in
original code);
                         <GenAI #1 you used>, <GenAI #2 you used> (Assignment 5 revisions)
Code authorship key used in comments: [ORIG] = Peter Lauman / ChatGPT, unchanged;
                                      [REV]  = added or changed for Assignment 5 (GenAI-assisted, then
edited by Nathaniel Lanter)
*/
#include <array>      // [ORIG] Fixed-size containers hold the nine-by-nine grid.
#include <fstream>    // [ORIG] File streams read puzzles; [REV] also used to write solution files.
#include <iostream>   // [ORIG] Standard output displays puzzles and solutions.
#include <ostream>    // [REV] Lets print() send a grid to either the console or a file.
#include <string>     // [ORIG] File names and error messages are strings.
#include <vector>     // [ORIG] Candidate lists store valid numbers for a cell.

class SudokuSolver {
private:
   std::array<std::array<int, 9>, 9> grid{}; // [ORIG] Zero represents an empty square.
   unsigned long long solutions = 0;        // [ORIG] Track how many completed grids are found.
   std::ofstream solutionFile;              // [REV] Output stream for this puzzle's solutionN.txt.

   // [ORIG] Check whether a number violates any Sudoku constraint at a given square.
   bool allowed(int row, int col, int value) const {
      for (int i = 0; i < 9; ++i) { // [ORIG] Scan the corresponding row and column.
         if (grid[row][i] == value || grid[i][col] == value) return false; // [ORIG] Digit already in
row/column.
      }
```

```
        int firstRow = (row / 3) * 3; // [ORIG] Find the top row of the containing block.
        int firstCol = (col / 3) * 3; // [ORIG] Find the left column of the containing block.
        for (int r = firstRow; r < firstRow + 3; ++r) { // [ORIG] Visit each block row.
            for (int c = firstCol; c < firstCol + 3; ++c) { // [ORIG] Visit each block column.
                if (grid[r][c] == value) return false; // [ORIG] Reject repeated digits.
            }
        }
        return true; // [ORIG] No conflict was found.
    }

    // [ORIG] Use the empty square with the fewest legal candidates to reduce branching.
    bool chooseCell(int& bestRow, int& bestCol, std::vector<int>& bestValues) const {
        bestRow = -1; // [ORIG] Negative row means the board has no empty cells.
        bestCol = -1; // [ORIG] Negative column also indicates no choice yet.
        bestValues.clear(); // [ORIG] Remove candidates from the previous recursion level.
        for (int r = 0; r < 9; ++r) { // [ORIG] Inspect all rows.
            for (int c = 0; c < 9; ++c) { // [ORIG] Inspect all columns.
                if (grid[r][c] != 0) continue; // [ORIG] Ignore already filled cells.
                std::vector<int> candidates; // [ORIG] Store legal values for this square.
                for (int v = 1; v <= 9; ++v) { // [ORIG] Try digits one through nine.
                    if (allowed(r, c, v)) candidates.push_back(v); // [ORIG] Keep legal digits.
                }
                if (candidates.empty()) { // [ORIG] An empty square has no valid digit.
                    bestRow = r; // [ORIG] Identify the contradictory square.
                    bestCol = c; // [ORIG] Identify its column.
                    bestValues.clear(); // [ORIG] Signal that this branch cannot work.
                    return true; // [ORIG] A blank cell exists, but this branch is impossible.
                }
                if (bestRow == -1 || candidates.size() < bestValues.size()) { // [ORIG] Prefer fewer
options.
                    bestRow = r; // [ORIG] Save the strongest next square.
                    bestCol = c; // [ORIG] Save its column.
                    bestValues = candidates; // [ORIG] Save its available digits.
                }
            }
        }
        return bestRow != -1; // [ORIG] False means the entire board is filled.
    }

    // [ORIG] Recursively explore every valid completion of the current grid.
    void backtrack() {
        int row, col; // [ORIG] Coordinates of the next empty square.
        std::vector<int> values; // [ORIG] Valid candidates for that square.
        if (!chooseCell(row, col, values)) { // [ORIG] No empty squares remain.
```

```
            ++solutions; // [ORIG] Count the completed Sudoku board.
            std::cout << "Solution " << solutions << ":\n"; // [ORIG] Label each solution on the
console.
            print(std::cout); // [REV] Print the completed board to the console (print now takes a
stream).
            print(solutionFile); // [REV] Also write the completed board to solutionN.txt in the input
format.
            return; // [ORIG] Continue searching after returning to the caller.
        }
        for (int value : values) { // [ORIG] Explore each candidate in order.
            grid[row][col] = value; // [ORIG] Tentatively fill the chosen square.
            backtrack(); // [ORIG] Recursively solve the smaller puzzle.
            grid[row][col] = 0; // [ORIG] Undo the guess so other possibilities can be tried.
        }
    }

public:
    // [ORIG] Read exactly 81 tokens, allowing underscores for blank squares.
    bool load(const std::string& filename) {
        std::ifstream input(filename); // [ORIG] Open the named puzzle.
        if (!input) return false; // [ORIG] Report missing or unreadable files.
        grid = {}; // [ORIG] Reset the grid before reading another puzzle.
        solutions = 0; // [ORIG] Reset the solution counter.
        std::string token; // [ORIG] Hold each number or underscore.
        for (int r = 0; r < 9; ++r) { // [ORIG] Read nine rows.
            for (int c = 0; c < 9; ++c) { // [ORIG] Read nine cells per row.
                if (!(input >> token)) return false; // [ORIG] Reject short or malformed files.
                if (token == "_") grid[r][c] = 0; // [ORIG] Convert a blank to zero.
                else if (token.size() == 1 && token[0] >= '1' && token[0] <= '9') // [ORIG] Accept a
single digit 1-9.
                    grid[r][c] = token[0] - '0'; // [ORIG] Convert a digit to an integer.
                else return false; // [ORIG] Reject unexpected input.
            }
        }
        if (input >> token) return false; // [ORIG] Reject files containing extra cells.
        for (int r = 0; r < 9; ++r) { // [ORIG] Check all given values.
            for (int c = 0; c < 9; ++c) { // [ORIG] Check every square in each row.
                if (grid[r][c] == 0) continue; // [ORIG] Blank squares cannot conflict.
                int value = grid[r][c]; // [ORIG] Temporarily remove a given digit.
                grid[r][c] = 0; // [ORIG] Avoid comparing the given with itself.
                bool valid = allowed(r, c, value); // [ORIG] Test row, column, and block.
                grid[r][c] = value; // [ORIG] Restore the original given digit.
                if (!valid) return false; // [ORIG] Reject conflicting clues.
            }
```

```
        }
        return true; // [ORIG] The puzzle was read and validated.
    }

    // [REV] Print the grid to any output stream (console or file) in the same format as the input
files.
    void print(std::ostream& out) const {
        for (const auto& row : grid) { // [ORIG] Iterate through the nine rows.
            for (int c = 0; c < 9; ++c) { // [ORIG] Print all nine columns.
                if (c) out << ' '; // [REV] Separate adjacent cells with spaces (now writes to "out").
                if (row[c] == 0) out << '_'; // [REV] Show blanks as underscores.
                else out << row[c]; // [REV] Show filled squares as digits.
            }
            out << '\n'; // [REV] End the current row.
        }
        out << '\n'; // [REV] Blank line separates grids visually.
    }

    // [REV] Solve the loaded puzzle and write all results to the named solution file.
    void solve(const std::string& solutionName) {
        solutionFile.open(solutionName); // [REV] Create/overwrite solutionN.txt for this puzzle.
        backtrack(); // [ORIG] Begin recursive depth-first search.
        if (solutions == 0) { // [ORIG] Handle failure.
            std::cout << "No solution found\n"; // [ORIG] Console message for unsolvable boards.
            solutionFile << "No solution found.\n"; // [REV] File must contain exactly this single line.
        } else { // [REV] At least one solution was found.
            std::cout << "Total solutions: " << solutions << "\n"; // [ORIG] Report count on the
console.
            solutionFile << "Solutions found: " << solutions << "\n"; // [REV] Required final line in the
file.
        }
        solutionFile.close(); // [REV] Flush and close the file so it is fully written.
    }
};

// [ORIG] Run the same object-oriented solver on all five assigned puzzle files.
int main() {
    for (int number = 1; number <= 5; ++number) { // [ORIG] Process every supplied puzzle.
        std::string filename = "puzzle" + std::to_string(number) + ".txt"; // [ORIG] Build file name.
        std::string solutionName = "solution" + std::to_string(number) + ".txt"; // [REV] Build output
file name.
        SudokuSolver sudoku; // [ORIG] Create a fresh object for this puzzle.
        std::cout << "=== " << filename << " ===\n"; // [ORIG] Identify the input file.
        if (!sudoku.load(filename)) { // [ORIG] Read and validate the puzzle.
```

```
            std::cout << "Unable to read puzzle or invalid starting grid.\n\n"; // [ORIG] Explain failure.
            std::ofstream bad(solutionName); // [REV] Still create a solution file for this puzzle.
            bad << "No solution found.\n"; // [REV] An unusable puzzle has no solution.
            continue; // [ORIG] Move on to the next puzzle file.
        }
        std::cout << "Original puzzle:\n"; // [ORIG] Label the given grid.
        sudoku.print(std::cout); // [REV] Show the initial board (print now takes a stream).
        sudoku.solve(solutionName); // [REV] Find all solutions and write them to solutionN.txt.
        std::cout << '\n'; // [ORIG] Separate results from different puzzles.
    }
    return 0; // [ORIG] Finish successfully.
}
```